Bachelor Intro to CV – week 7 exercise tasks

Week 07: Fine-Tuning of neural nets

Task1:

Macht die Aufgaben aus der Vorlesung zu Kettenregel und zu einsum, falls noch nicht geschehen.

Task2: Training from scratch vs fine-tuning.

- take the 102 class flowers dataset and write a dataset class which can work with a train/val/test split for it (named trainfile.txt, valfile.txt and just testfile.txt .).

The difference to the 102 flowers from oxford is that I provided in it for you train/val/test split for your convenience (bowing politely in front of the students).

- take any deep network you like which has pretrained weights (a small efficientnet, a mobilenet are training fast)
- train a deep neural network in three different modes . For small nets you can do this on a reasonable CPU. A GPU is not needed.
  - (A) once without loading weights and training all layers.
  - (B) once with loading model weights before training and training all layers,
  - (C) once with loading model weights before training and training only the **last two trainable** layers,
- For each of these modes select the best model by measuring in every epoch the performance of the model on the validation set. Save that best model
- measure the performance of the selected best model once on the test set

- other notes: Typically less than 15 epochs should suffice for training when using finetuning. You can get to similar performance without loading weights using more epochs and more data augmentation – with a higher effort.

  You can run also optionally a selection over a few learning rates, if you use a GPU.

What do you need to do for steps when you start with a code like the MNIST training code?

- write a new dataloader for your training dataset, the flowers. To do this, you need to implement the two interfaces, and apply the transforms after loading an image. The train/val/test splitfiles are provided (see above).
- adjust paths for data (and if necessary for label paths/files or files determining splits into train/val/test)
- write the code so that it works with paths relative to the directory of your script
- for data augmentation you can simply start with the following:

```
data_transforms = {
    'train': transforms.Compose([
        transforms.Resize(256),
        transforms.RandomCrop(224),
        #further augmentations here or after the totensor
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}
```

I am aware that I am using here the imagenet mean and the imagenet standard deviation. You can compute the dataset specific standard deviation of course.

- use some deep learning model from the model zoo, load its weights before training for settings B and C
- adjust the number of classes in the last linear/dense layer
- in the optimizer provide the proper parameters for training for settings A,B,C

- collect and plot curves of the training loss, the validation set loss and the validation set accuracy. also print the final test accuracy of the selected model
- A note: You can check `https://github.com/pytorch/vision/blob/master/torchvision/models/resnet.py` to see what routine is used to load model weights